

BLACKBOXAI

Generated on: 9/3/2025 10:04:59 PM

User

how to resolve ssl certificate issue

BLACKBOXAI

SSL Certificate Issue Resolution

The log file indicates multiple instances of "The SSL certificate has expired" errors. To resolve this, you need to update the expired SSL certificates.

1. Identify Expired Certificates:

- The log entries clearly state which certificates have expired, for example:

Subject: EMAILADDRESS=premium-server@thawte.com, CN=Thawte Premium Server CA Subject: CN=Equifax Secure eBusiness CA-1, O=Equifax Secure Inc., C=US Subject: CN=UTN-USERFirst-Client Authentication and Email, OU=http://www.usertrust.com, O=The USERTRUST Network, L=Salt Lake City, ST=UT, C=US And many others listed with NCLS-SECURITY-05054 message.

2. Obtain New Certificates:

- Contact the Certificate Authorities (CAs) mentioned in the "Subject" field of the expired certificates (e.g., Thawte, Equifax Secure Inc., The USERTRUST Network, GlobalSign, Certplus, Entrust.net, etc.) to obtain new, valid certificates.
- Alternatively, if these are self-signed certificates or certificates from an internal CA, you will need to regenerate them.

3. Replace Expired Certificates:

- The log indicates that the trust store is located at `/opt/payara41/glassfish/domains/domain1/config/cacerts.jks` and the keystore at `/opt/payara41/glassfish/domains/domain1/config/keystore.jks`.
- You will need to import the newly obtained certificates into the appropriate keystore or trust store. The exact commands will depend on the format of the new certificates (e.g., JKS, PKCS#12) and the tool you use (e.g., `keytool`).

General Steps for Replacing Certificates using `keytool` (assuming JKS format):

- Backup: Before making any changes, back up your existing `cacerts.jks` and `keystore.jks` files.
- Import Root/Intermediate Certificates (into `cacerts.jks`):

Generated by BlackBox AI

```
bash
keytool -import -trustcacerts -alias [alias_name] -file [certificate_file.cer] -
keystore /opt/payara41/glassfish/domains/domain1/config/cacerts.jks
```

Replace `[alias_name]` with a unique name for the certificate. Replace `[certificate_file.cer]` with the path to your root or intermediate CA certificate file.

- Import Server Certificate (into `keystore.jks`):

```
bash
keytool -importkeystore -srckeystore [pfx_file.p12] -srcstoretype PKCS12 -
destkeystore /opt/payara41/glassfish/domains/domain1/config/keystore.jks -
deststoretype JKS
```

If your new server certificate is in PKCS#12 format (`.p12` or `.pfx`), use this command. If you have separate key and certificate files (e.g., `.key` and `.crt`), you might need to combine them into a PKCS#12 file first or import them directly if `keytool` supports it.

- Verify: After importing, you can list the contents of the keystore/truststore to verify the new certificates:

```
bash
keytool -list -keystore
/opt/payara41/glassfish/domains/domain1/config/cacerts.jks
keytool -list -keystore
/opt/payara41/glassfish/domains/domain1/config/keystore.jks
```

4. Restart Payara Server:

- After updating the certificates, restart the Payara Server to ensure the changes take effect.

Additional Considerations:

- **SSL Cipher Suites:** The log also shows warnings about "All SSL cipher suites disabled for network-listener(s). Using SSL implementation specific defaults." While not directly causing the expired certificate issue, this indicates a potential security weakness. You should review and configure appropriate SSL/TLS protocols and cipher suites in your Payara Server configuration to ensure secure communication.
- **Database Connection:** The log shows a `SocketTimeoutException` when connecting to `dbstst02.brain2.com:5432` . This is a separate issue indicating that the Payara server cannot establish a connection to the database. This could be due to:

Incorrect database host or port. Firewall rules blocking the connection. The database server not running or not accessible from the Payara server. Network connectivity issues. You

need to investigate the network connectivity between the Payara server and the database server (dbstst02.brain2.com on port 5432).

User

From where I will get new SSL certificate for payara below is my dockerfile FROM payara/server-full:181

```
ARG ENV ARG CUSTOM_PAYARA_ARGS ARG ARTIFACT_NAME
LABEL environment=${ENV} LABEL app-version=${APP_VERSION}
ENV DOMAIN_HOME=${PAYARA_PATH}/glassfish/domains/${PAYARA_DOMAIN} ENV
PAYARA_ARGS=${CUSTOM_PAYARA_ARGS} ENV
MONTOVA_BASE_DIR=/opt/montova ENV
MONTOVA_INIT_DIR=${MONTOVA_BASE_DIR}/init ENV PASSWORD_FILE=/opt/pwdfile
USER root
```

Create folders

```
RUN mkdir -p /deploy ${MONTOVA_BASE_DIR} ${MONTOVA_INIT_DIR}
COPY bin/.sh ${MONTOVA_INIT_DIR}/ RUN chmod +x ${MONTOVA_INIT_DIR}/.sh
```

Copy content to container

```
COPY profiles/${ENV}/post-boot-commands.asadmin /tmp COPY
profiles/common/additonal-post-boot-commands.asadmin /tmp

RUN echo "deb [trusted=yes] http://archive.debian.org/debian stretch main non-free
contrib" > /etc/apt/sources.list && echo "deb-src [trusted=yes]
http://archive.debian.org/debian stretch main non-free contrib" >> /etc/apt/sources.list &&
echo "deb [trusted=yes] http://archive.debian.org/debian-security stretch/updates main non-
free contrib" >> /etc/apt/sources.list && echo 'Acquire::Check-Valid-Until "false";' >
/etc/apt/apt.conf.d/99no-check-valid-until

RUN apt-get update RUN apt-get -y upgrade RUN apt-get install -y netcat RUN apt-get
install -y telnet RUN apt-get install -y dos2unix RUN dos2unix ${MONTOVA_INIT_DIR}/.sh

RUN rm -f /etc/localtime RUN ln -s /usr/share/zoneinfo/Europe/Brussels /etc/localtime RUN
dpkg-reconfigure -f noninteractive tzdata
```

Prepare post boot commands

Append our custom post boot command file to the existing one

```
RUN sed -i "s@%ARTIFACT_NAME%@$ARTIFACT_NAME@g" /tmp/post-boot-
commands.asadmin RUN cat /tmp/post-boot-commands.asadmin >>
```

```
${POSTBOOT_COMMANDS} RUN cat /tmp/additonal-post-boot-commands.asadmin >>
${POSTBOOT_COMMANDS}
```

Copy classes

```
COPY profiles/${ENV}/libs/classes/ ${DOMAIN_HOME}/lib/classes/
```

Copy libs to domain lib

```
COPY profiles/common/libs/ext/.jar ${DOMAIN_HOME}/lib/ext
```

Copy artifact(s)

```
COPY artifacts/${ARTIFACT_NAME} /deploy
```

Link the logs to our default log location

```
RUN ln -s ${DOMAIN_HOME}/logs/ /log
```

Cleanup temporary data

```
RUN chown -R payara:payara ${PAYARA_PATH}
```

```
EXPOSE 4848 8080 9009 5432
```

```
ENTRYPOINT ["/opt/montova/init/entrypoint.sh"]
```

BLACKBOXAI

Based on your Dockerfile and the previous log analysis, here's a breakdown of where you might get new SSL certificates for Payara:

1. Understanding the Certificates in Your Logs

The "expired SSL certificate" errors in your `payaraLogs.txt` are primarily related to root and intermediate CA certificates within Payara's trust store (`cacerts.jks`). These are certificates that Payara uses to trust other entities (like external services, or even its own internal components if they use certificates signed by these CAs).

The specific expired certificates mentioned are:

- Thawte Premium Server CA
 - Thawte Server CA
 - Equifax Secure eBusiness CA-1
 - UTN-USERFirst-Client Authentication and Email
 - Thawte Personal Freemail CA
 - UTN-USERFirst-Hardware
-
- AddTrust Class 1 CA Root

- Equifax Secure Certificate Authority
- ValiCert Class 1 Policy Validation Authority
- Equifax Secure Global eBusiness CA-1
- Deutsche Telekom Root CA 2
- UTN-USERFirst-Object
- Baltimore CyberTrust Code Signing Root
- Baltimore CyberTrust Root
- Class 3P Primary CA (Certplus)
- UTN - DATACorp SGC
- GTE CyberTrust Global Root
- Security Communication RootCA1
- GlobalSign
- Class 2 Primary CA (Certplus)
- GeoTrust Global CA
- Sonera Class2 CA
- Thawte Timestamping CA
- Sonera Class1 CA
- QuoVadis Root Certification Authority
- ValiCert Class 2 Policy Validation Authority
- AddTrust Qualified CA Root
- KEYNECTIS ROOT CA
- AddTrust External CA Root
- Entrust.net Certification Authority (2048)
- Entrust.net Secure Server Certification Authority

These are well-known public Certificate Authorities. The fact that so many of them are expired suggests that the `cacerts.jks` file in your Payara image is outdated.

2. Sources for New SSL Certificates

You'll need to distinguish between two types of certificates:

- Root/Intermediate CA Certificates (for `cacerts.jks`):

OpenJDK/Java Distribution: The most common source for an updated `cacerts.jks` is the Java Runtime Environment (JRE) or Java Development Kit (JDK) itself. The `cacerts` file is part of the Java installation and contains a collection of trusted root certificates. Operating System Updates: Sometimes, the underlying operating system (Debian Stretch in your case) might provide updated CA certificate bundles, which Java might then pick up or which you could manually import. Direct from CAs: You can download individual root and intermediate certificates directly from the websites of the respective Certificate Authorities

Generated by BlackBox AI

(e.g., DigiCert, Sectigo, Let's Encrypt, etc.). However, for a comprehensive set, relying on an updated Java distribution is usually easier.

- Your Application's Specific Certificates (for `keystore.jks` - if applicable):

If your application (e.g., `workflow-ear-3.1.2-SNAPSHOT.ear`) uses its own SSL certificates for inbound connections (e.g., HTTPS for a web interface) or for outbound connections that require client authentication, these certificates would be in your `keystore.jks`. You would obtain these from a commercial CA (like those listed above, but you'd be the subscriber, not just trusting their root) or generate them yourself if they are for internal use.

3. How to Get Them into Your Docker Image

Given your Dockerfile, here are the primary strategies:

A. Update the Base Payara Image (Recommended for `cacerts.jks`)

- The `payara/server-full:181` image is based on a specific version of Payara and its underlying Java. Version 181 is quite old (likely from 2018). The expired certificates strongly suggest that the Java version within this image has an outdated `cacerts.jks`.
- Solution: The simplest and most robust solution is to update your base image to a newer Payara version. Newer Payara images will come with a more recent Java version, which in turn will have an updated `cacerts.jks` file.

Example: `FROM payara/server-full:latest` OR `FROM payara/server-full:6.2023.10-full` (or whatever the latest stable version is when you build).

- Benefit: This automatically updates the entire set of trusted root certificates, addressing the bulk of your "expired SSL certificate" errors.

B. Manually Update `cacerts.jks` within the Dockerfile

If you cannot update the base image, you can try to update the `cacerts.jks` file:

- Find a Newer `cacerts.jks`:

Download a recent JDK/JRE (e.g., OpenJDK 8, 11, or 17) from a reputable source (e.g., Adoptium, Oracle). Locate its `cacerts` file (usually in `[JDK_HOME]/jre/lib/security/cacerts` or `[JRE_HOME]/lib/security/cacerts`).

- Copy and Replace in Dockerfile:

```
dockerfile
# ... (your existing Dockerfile content) ...

# Temporarily install Java to get its cacerts, or copy from build context
# Option 1: Install a newer JRE/JDK just to get the cacerts file
RUN apt-get update && apt-get install -y openjdk-11-jre-headless &&
    cp /etc/ssl/certs/java/cacerts ${JAVA_HOME}/lib/security/cacerts &&
    apt-get remove --purge -y openjdk-11-jre-headless && apt-get clean && rm -rf
/var/lib/apt/lists/
```

Generated by BlackBox AI

```
# Option 2: Copy a cacerts file from your build context (if you download it manually)
# COPY path/to/your/downloaded/cacerts ${JAVA_HOME}/lib/security/cacerts

# Ensure correct permissions and ownership
RUN chown payara:payara ${JAVA_HOME}/lib/security/cacerts

# ... (rest of your Dockerfile) ...
```

Note: The `JAVA_HOME` environment variable might not be set in your base image, or it might point to a different location. You'll need to inspect the `payara/server-full:181` image to find the correct path to its `lib/security` directory. You can do this by running `docker run --rm -it payara/server-full:181 bash` and then using `find / -name cacerts`. This method is more complex and might introduce inconsistencies if the new `cacerts` is from a significantly different Java version than what Payara 181 expects.

C. For Your Application's `keystore.jks` (if applicable)

- If your application (`workflow-ear-3.1.2-SNAPSHOT.ear`) requires its own SSL certificate (e.g., for an HTTPS endpoint), you would typically generate a Certificate Signing Request (CSR) from your `keystore.jks` , send it to a CA, and then import the signed certificate back into that `keystore.jks` .
- Process:
 1. Generate Key Pair and CSR: Use `keytool` to generate a new key pair and a CSR.
 2. Submit CSR to CA: Send the CSR to your chosen Certificate Authority (e.g., Let's Encrypt, DigiCert, Sectigo, etc.).
 3. Receive Signed Certificate: The CA will provide you with your signed certificate and possibly intermediate certificates.
 4. Import into `keystore.jks` : Use `keytool` to import the root, intermediate, and your signed certificate into your `keystore.jks` .
- Docker Integration:

You would then `COPY` this updated `keystore.jks` file into your Docker image. Example:

```
dockerfile
# ... (your existing Dockerfile content) ...

# Copy your application's keystore (after it's updated with new certs)
COPY profiles/${ENV}/config/keystore.jks ${DOMAIN_HOME}/config/keystore.jks

# ... (rest of your Dockerfile) ...
```

Ensure the `javax.net.ssl.keyStore` system property (seen in your logs) points to this updated file.

Recommendation:

Start by updating your base Payara image to the latest stable version. This is the cleanest and most effective way to resolve the widespread expired CA certificate issue. If that doesn't fully resolve it or if your application has its own specific certificates, then proceed with manually updating `cacerts.jks` or `keystore.jks` as needed.

User

I dont want to update payara base image. From where I will get the certificates should I downloaded it from new payara version

BLACKBOXAI

Okay, if you absolutely cannot update the Payara base image (`payara/server-full:181`), then you'll need to manually update the `cacerts.jks` file within your existing image.

Where to Get the Certificates (specifically, an updated `cacerts.jks`):

You are correct in thinking about getting it from a newer Payara version or a newer Java distribution. The `cacerts.jks` file is part of the Java Runtime Environment (JRE) and contains the trusted root certificates.

Here are the best sources:

- 1. From a Newer Payara Image (Recommended Approach if not updating base image):

Newer Payara images (e.g., `payara/server-full:latest` or a recent version like `payara/server-full:6.2023.10-full`) will contain a more up-to-date JRE with a current `cacerts.jks` . How to get it: Run a temporary container of a newer Payara image:

```
bash
docker run --rm -it --name temp_payara payara/server-full:latest bash
```

Once inside the container, locate the `cacerts.jks` file. It's usually in `$JAVA_HOME/lib/security/cacerts` . You can find `$JAVA_HOME` by running `echo $JAVA_HOME` or search for it: `find / -name cacerts.jks 2>/dev/null` . Copy the file out of the container to your local machine:

```
bash
docker cp temp_payara:/path/to/cacerts.jks ./new_cacerts.jks
```

(Replace `/path/to/cacerts.jks` with the actual path you found inside the container). Stop the temporary container:

```
bash
docker stop temp_payara
```

- 2. From a Recent OpenJDK Distribution:

You can download a standalone OpenJDK JRE (e.g., OpenJDK 8, 11, or 17) from a reputable provider like Adoptium (formerly AdoptOpenJDK). How to get it: Go to the Adoptium website (e.g., <https://adoptium.net/temurin/releases/>). Download a recent JRE for Linux (x64). Extract the downloaded archive. Navigate to `[extracted_folder]/lib/security/` and you will find `cacerts`. Benefit: This gives you direct control over the source of the `cacerts` file.

Once you have the `new_cacerts.jks` file:

You will need to integrate this file into your existing Dockerfile.

Updated Dockerfile Snippet:

```
dockerfile
FROM payara/server-full:181

ARG ENV
ARG CUSTOM_PAYARA_ARGS
ARG ARTIFACT_NAME

LABEL environment=${ENV}
LABEL app-version=${APP_VERSION}

ENV DOMAIN_HOME=${PAYARA_PATH}/glassfish/domains/${PAYARA_DOMAIN}
ENV PAYARA_ARGS=${CUSTOM_PAYARA_ARGS}
ENV MONTOVA_BASE_DIR=/opt/montova
ENV MONTOVA_INIT_DIR=${MONTOVA_BASE_DIR}/init
ENV PASSWORD_FILE=/opt/pwdfile

USER root
```

Create folders

```
RUN mkdir -p /deploy ${MONTOVA_BASE_DIR} ${MONTOVA_INIT_DIR}
COPY bin/.sh ${MONTOVA_INIT_DIR}/
RUN chmod +x ${MONTOVA_INIT_DIR}/.sh
```

--- START: Section to update cacerts.jks ---

First, find the current `JAVA_HOME` or the location of `cacerts.jks` in your 181 image.

You can do this by running: `docker run --rm -it payara/server-full:181 bash`

Then inside: `echo $JAVA_HOME` or `find / -name cacerts.jks 2>/dev/null`

Let's assume it's /opt/payara41/jdk/jre/lib/security/cacerts for this example,

but you MUST verify this path in your specific 181 image.

```
ENV PAYARA_JAVA_SECURITY_DIR=${PAYARA_PATH}/jdk/jre/lib/security # Adjust this path if needed!
```

Copy the newly obtained cacerts.jks from your build context

Make sure 'new_cacerts.jks' is in the same directory as your Dockerfile,

or adjust the COPY source path accordingly.

```
COPY new_cacerts.jks ${PAYARA_JAVA_SECURITY_DIR}/cacerts
```

Ensure correct permissions and ownership for the new cacerts file

```
RUN chown payara:payara ${PAYARA_JAVA_SECURITY_DIR}/cacerts
RUN chmod 644 ${PAYARA_JAVA_SECURITY_DIR}/cacerts
```

--- END: Section to update cacerts.jks ---

```
COPY profiles/${ENV}/post-boot-commands.asadmin /tmp
COPY profiles/common/additonal-post-boot-commands.asadmin /tmp

RUN echo "deb [trusted=yes] http://archive.debian.org/debian stretch main non-free contrib" > /etc/apt/sources.list &&
    echo "deb-src [trusted=yes] http://archive.debian.org/debian stretch main non-free contrib" >> /etc/apt/sources.list &&
    echo "deb [trusted=yes] http://archive.debian.org/debian-security stretch/updates main non-free contrib" >> /etc/apt/sources.list &&
    echo 'Acquire::Check-Valid-Until "false";' > /etc/apt/apt.conf.d/99no-check-valid-until

RUN apt-get update
RUN apt-get -y upgrade
RUN apt-get install -y netcat
RUN apt-get install -y telnet
RUN apt-get install -y dos2unix
RUN dos2unix ${MONTOVA_INIT_DIR}/.sh

RUN rm -f /etc/localtime
RUN ln -s /usr/share/zoneinfo/Europe/Brussels /etc/localtime
```

Generated by BlackBox AI

```
RUN dpkg-reconfigure -f noninteractive tzdata
```

Prepare post boot commands

Append our custom post boot command file to the existing one

```
RUN sed -i "s@%ARTIFACT_NAME%@$ARTIFACT_NAME@g" /tmp/post-boot-commands.asadmin  
RUN cat /tmp/post-boot-commands.asadmin >> ${POSTBOOT_COMMANDS}  
RUN cat /tmp/additonal-post-boot-commands.asadmin >> ${POSTBOOT_COMMANDS}
```

Copy classes

```
COPY profiles/${ENV}/libs/classes/ ${DOMAIN_HOME}/lib/classes/
```

Copy libs to domain lib

```
COPY profiles/common/libs/ext/.jar ${DOMAIN_HOME}/lib/ext
```

Copy artifact(s)

```
COPY artifacts/${ARTIFACT_NAME} /deploy
```

Link the logs to our default log location

```
RUN ln -s ${DOMAIN_HOME}/logs/ /log
```

Cleanup temporary data

```
RUN chown -R payara:payara ${PAYARA_PATH}
```

```
EXPOSE 4848 8080 9009 5432
```

```
ENTRYPOINT ["/opt/montova/init/entrypoint.sh"]
```

Important Verification Step:

Before building, you must verify the exact path to the `cacerts.jks` file within your `payara/server-full:181` image.

- 1. Run your current image interactively:

```
bash  
docker run --rm -it payara/server-full:181 bash
```

- 2. Inside the container, try these commands to find the `cacerts.jks` file:

Generated by BlackBox AI

```
bash
echo $JAVA_HOME
find / -name cacerts.jks 2>/dev/null
```

The output of `find` will give you the precise path. Update the `ENV` `PAYARA_JAVA_SECURITY_DIR` line in the Dockerfile with the directory containing `cacerts.jks`. For example, if `find` returns `/usr/lib/jvm/java-8-openjdk-amd64/jre/lib/security/cacerts`, then `PAYARA_JAVA_SECURITY_DIR` should be `/usr/lib/jvm/java-8-openjdk-amd64/jre/lib/security`.

By following these steps, you can update the trusted root certificates without changing your base Payara image version.